

CAPSTONE PROJECT AUTOMATED FINANCE DASHBOARD FOR SMALL BUSINESS

STEP 1 — Import pandas

```
import pandas as pd
```

STEP 2 — Load the CSV file

```
df = pd.read_csv("stock_analysis_100_records.csv")  
  
df
```

```
[4]:
```

	Date	Open	High	Low	Close	Volume
0	2026-01-05	100.00	100.34	99.06	100.00	1035672
1	2026-01-06	99.15	100.87	97.93	99.85	610835
2	2026-01-07	99.60	102.35	99.35	101.12	760093
3	2026-01-08	100.91	104.43	99.89	103.99	850786
4	2026-01-09	103.49	104.70	102.45	103.65	857047
...	...	...	...	...	...	...
95	2026-05-18	87.80	88.30	85.11	85.49	1078468
96	2026-05-19	85.69	86.89	85.45	86.03	930836
97	2026-05-20	85.57	87.85	84.48	86.52	694413
98	2026-05-21	86.60	87.14	86.50	86.61	821449
99	2026-05-22	86.64	87.81	85.68	86.34	718657

100 rows × 6 columns

STEP 3 — Check rows and columns

```
df.shape
```

(100, 6)

df.head()

```
[6]:
```

	Date	Open	High	Low	Close	Volume
0	2026-01-05	100.00	100.34	99.06	100.00	1035672
1	2026-01-06	99.15	100.87	97.93	99.85	610835
2	2026-01-07	99.60	102.35	99.35	101.12	760093
3	2026-01-08	100.91	104.43	99.89	103.99	850786
4	2026-01-09	103.49	104.70	102.45	103.65	857047

STEP 4 — Basic data inspection

4.1 Display the last 3 records

df.tail(3)

```
[7]:
```

	Date	Open	High	Low	Close	Volume
95	2026-05-18	87.80	88.30	85.11	85.49	1078468
96	2026-05-19	85.69	86.89	85.45	86.03	930836
97	2026-05-20	85.57	87.85	84.48	86.52	694413
98	2026-05-21	86.60	87.14	86.50	86.61	821449
99	2026-05-22	86.64	87.81	85.68	86.34	718657

4.2 Display information about the dataset

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 6 columns):
 #   Column  Non-Null Count  Dtype
---  -
 0   Date    100 non-null    object
 1   Open    100 non-null    float64
 2   High    100 non-null    float64
 3   Low     100 non-null    float64
 4   Close   100 non-null    float64
 5   Volume  100 non-null    int64
dtypes: float64(4), int64(1), object(1)
memory usage: 4.8+ KB
```

4.3 Display data types

df.dtypes

```
[9]: Date      object
     Open      float64
     High      float64
     Low       float64
     Close     float64
     Volume    int64
     dtype: object
```

STEP 5 — Check missing values

df.isnull().sum()

```
[10]: Date      0
      Open      0
      High      0
      Low       0
      Close     0
      Volume    0
      dtype: int64
```

STEP 6 — Check duplicate records

df.duplicated().sum()

```
[11]: np.int64(0)
```

STEP 7 — Statistical summary

```
df.describe()
```

[70]:		Date	Open	High	Low	Close	Volume	Daily_Return_%	P
	count	100	100.000000	100.000000	100.000000	100.000000	1.000000e+02	99.000000	
	mean	2026-03-14 12:00:00	91.649400	93.053800	90.054100	91.494600	9.284023e+05	-0.127672	
	min	2026-01-05 00:00:00	81.900000	85.340000	81.510000	82.380000	5.047110e+05	-6.799412	
	25%	2026-02-08 06:00:00	86.957500	88.287500	85.427500	86.617500	7.374320e+05	-1.084324	
	50%	2026-03-14 12:00:00	89.200000	90.225000	87.680000	88.860000	8.497950e+05	-0.100818	
	75%	2026-04-17 18:00:00	94.235000	94.987500	92.810000	94.190000	9.682808e+05	1.053069	
	max	2026-05-22 00:00:00	108.240000	109.240000	106.840000	108.290000	3.418687e+06	5.098325	

STEP 8 — Convert Date column

```
df["Date"] = pd.to_datetime(df["Date"])
```

```
df.dtypes
```

```
[13]: Date      datetime64[ns]
      Open      float64
      High      float64
      Low       float64
      Close     float64
      Volume     int64
      dtype: object
```

```
print("Starting Date:", df["Date"].min())
```

```
print("Ending Date:", df["Date"].max())
```

```
Starting Date: 2026-01-05 00:00:00
Ending Date: 2026-05-22 00:00:00
```

STEP 9 — Clean the numerical columns

```
numeric_columns = ["Open", "High", "Low", "Close", "Volume"]
```

```
for col in numeric_columns:
```

```
    df[col] = pd.to_numeric(df[col], errors="coerce")
```

```
print("Numeric columns cleaned successfully!")
```

```
Numeric columns cleaned successfully!
```

STEP 10 — Check invalid financial values

```
print("Negative Open values:", (df["Open"] < 0).sum())
```

```
print("Negative High values:", (df["High"] < 0).sum())
```

```
print("Negative Low values:", (df["Low"] < 0).sum())
```

```
print("Negative Close values:", (df["Close"] < 0).sum())
```

```
print("Negative Volume values:", (df["Volume"] < 0).sum())
```

```
Negative Open values: 0
Negative High values: 0
Negative Low values: 0
Negative Close values: 0
Negative Volume values: 0
```

STEP 11 — Sort data by Date

```
df = df.sort_values("Date").reset_index(drop=True)
```

```
df.head()
```

```
[17]:
```

	Date	Open	High	Low	Close	Volume
0	2026-01-05	100.00	100.34	99.06	100.00	1035672
1	2026-01-06	99.15	100.87	97.93	99.85	610835
2	2026-01-07	99.60	102.35	99.35	101.12	760093
3	2026-01-08	100.91	104.43	99.89	103.99	850786
4	2026-01-09	103.49	104.70	102.45	103.65	857047

STEP 12 — Create additional analysis columns

```
df["Daily_Return_%"] = df["Close"].pct_change() * 100
df["Price_Range"] = df["High"] - df["Low"]

df["20_Day_MA"] = df["Close"].rolling(20).mean()

df["Month"] = df["Date"].dt.to_period("M").astype(str)
df["Quarter"] = df["Date"].dt.to_period("Q").astype(str)

df["Year"] = df["Date"].dt.year

df.head()
```

[18]:	Date	Open	High	Low	Close	Volume	Daily_Return_%	Price_Range	20_Day_MA
0	2026-01-05	100.00	100.34	99.06	100.00	1035672	NaN	1.28	NaN
1	2026-01-06	99.15	100.87	97.93	99.85	610835	-0.150000	2.94	NaN
2	2026-01-07	99.60	102.35	99.35	101.12	760093	1.271908	3.00	NaN
3	2026-01-08	100.91	104.43	99.89	103.99	850786	2.838212	4.54	NaN
4	2026-01-09	103.49	104.70	102.45	103.65	857047	-0.326955	2.25	NaN

STEP 13 — BASIC FINANCIAL ANALYSIS

```
total_volume = df["Volume"].sum()

print("Total Trading Volume:", total_volume)
```

```
Total Trading Volume: 92840228
```

```
average_close = df["Close"].mean()

print("Average Closing Price:", average_close)
```

```
Average Closing Price: 91.4946
```

```
highest_close = df["Close"].max()
```

```
print("Highest Closing Price:", highest_close)
```

```
Highest Closing Price: 108.29
```

```
lowest_close = df["Close"].min()
```

```
print("Lowest Closing Price:", lowest_close)
```

```
Lowest Closing Price: 82.38
```

```
average_volume = df["Volume"].mean()
```

```
print("Average Daily Volume:", average_volume)
```

```
Average Daily Volume: 928402.28
```

```
first_close = df.iloc[0]["Close"]
```

```
print("First Closing Price:", first_close)
```

```
First Closing Price: 100.0
```

```
last_close = df.iloc[-1]["Close"]
```

```
print("Last Closing Price:", last_close)
```

```
Last Closing Price: 86.34
```

```
overall_change = (last_close / first_close - 1) * 100
```

```
print("Overall Change:", overall_change, "%")
```

```
Overall Change: -13.659999999999995 %
```

STEP 14 — GROUP BY SUMMARY 1: YEAR

Group-by 1 — Year:

```
year_summary = df.groupby("Year").agg(
```

```
    Trading_Days=("Close", "count"),
```

```
    Average_Close=("Close", "mean"),
```

```
    Highest_Close=("Close", "max"),
```

```
    Lowest_Close=("Close", "min"),
```

```
Total_Volume=("Volume", "sum"),
Average_Volume=("Volume", "mean")
).reset_index()
```

year_summary

[28]:	Year	Trading_Days	Average_Close	Highest_Close	Lowest_Close	Total_Volume	Average_Vo
0	2026	100	91.4946	108.29	82.38	92840228	9284

STEP 15 — GROUP BY SUMMARY 2: MONTH

```
month_summary = df.groupby("Month").agg(
    Trading_Days=("Close", "count"),
    Average_Close=("Close", "mean"),
    Highest_Close=("Close", "max"),
    Lowest_Close=("Close", "min"),
    Total_Volume=("Volume", "sum"),
    Average_Volume=("Volume", "mean")
).reset_index()
```

month_summary

[30]:	Month	Trading_Days	Average_Close	Highest_Close	Lowest_Close	Total_Volume	Average_Vo
0	2026-01	20	102.764500	108.29	94.70	17068127	8.534
1	2026-02	20	92.698500	97.29	87.26	20664226	1.033
2	2026-03	22	86.364545	89.27	82.38	20910484	9.504
3	2026-04	22	88.415455	93.88	83.20	20670537	9.395
4	2026-05	16	87.190000	89.72	85.49	13526854	8.454

STEP 16 — GROUP BY SUMMARY 3: QUARTER

```
quarter_summary = df.groupby("Quarter").agg(  
    Trading_Days=("Close", "count"),  
    Average_Close=("Close", "mean"),  
    Highest_Close=("Close", "max"),  
    Lowest_Close=("Close", "min"),  
    Total_Volume=("Volume", "sum"),  
    Average_Volume=("Volume", "mean")  
)
```

```
quarter_summary
```

[32]:	Quarter	Trading_Days	Average_Close	Highest_Close	Lowest_Close	Total_Volume	Average_Volume
0	2026Q1	62	93.698065	108.29	82.38	58642837	9458
1	2026Q2	38	87.899474	93.88	83.20	34197391	8999

STEP 17 — IMPORT GRAPH LIBRARIES

```
import matplotlib.pyplot as plt  
import seaborn as sns
```

STEP 18 — CREATE GRAPH 1

Closing Price Over Time

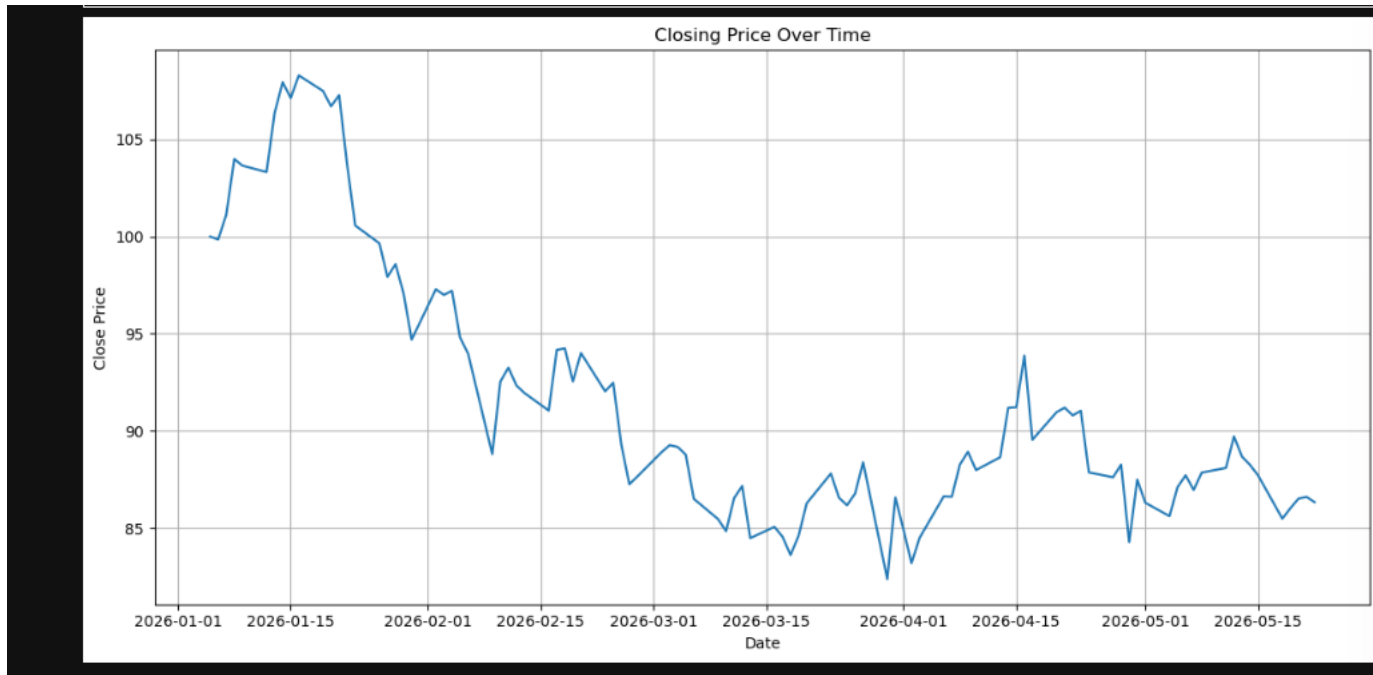
```
plt.figure(figsize=(12, 6))  
plt.plot(df["Date"], df["Close"])  
plt.title("Closing Price Over Time")  
plt.xlabel("Date")
```

```
plt.ylabel("Close Price")
```

```
plt.grid(True)
```

```
plt.tight_layout()
```

```
plt.show()
```



STEP 19 — CREATE GRAPH 2

OHLC Price Movement

```
plt.figure(figsize=(12, 6))
```

```
plt.plot(df["Date"], df["Open"], label="Open")
```

```
plt.plot(df["Date"], df["High"], label="High")
```

```
plt.plot(df["Date"], df["Low"], label="Low")
```

```
plt.plot(df["Date"], df["Close"], label="Close")
```

```
plt.title("OHLC Price Movement")
```

```
plt.xlabel("Date")
```

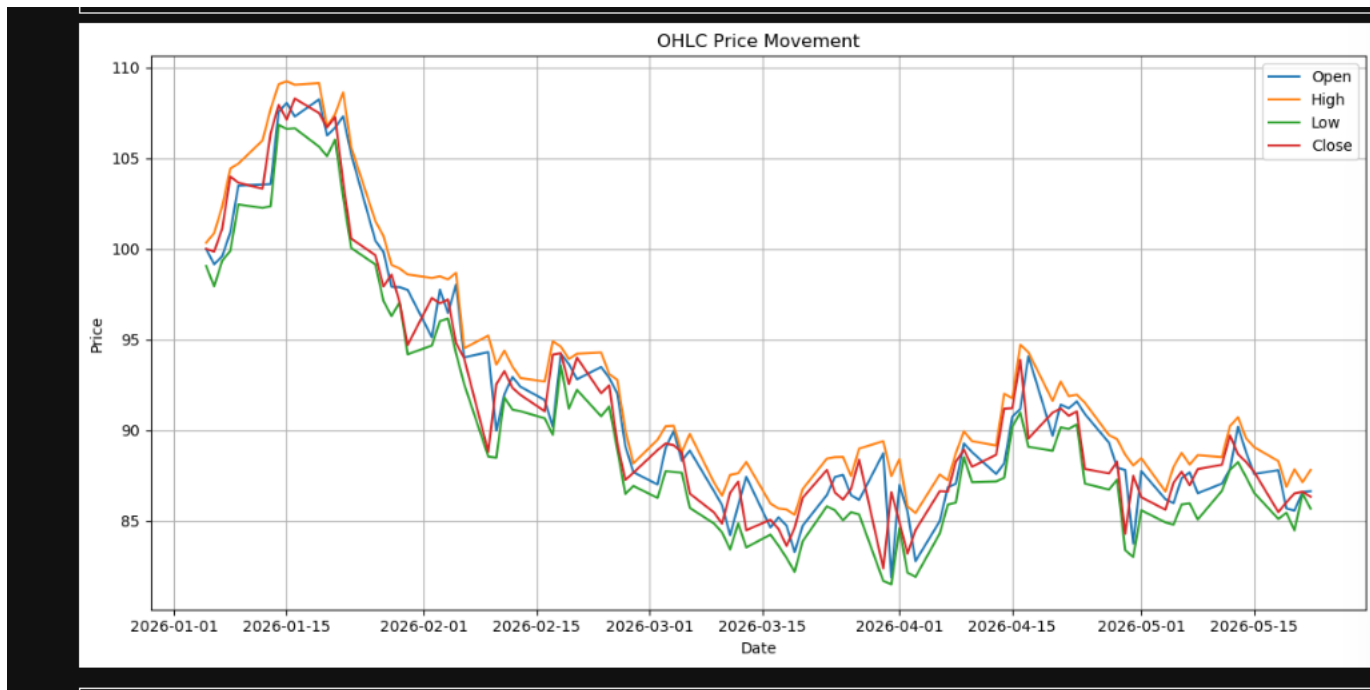
```
plt.ylabel("Price")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.tight_layout()
```

```
plt.show()
```



STEP 20 — CREATE GRAPH 3

Daily Trading Volume

```
plt.figure(figsize=(12, 6))
```

```
plt.bar(df["Date"], df["Volume"])
```

```
plt.title("Daily Trading Volume")
```

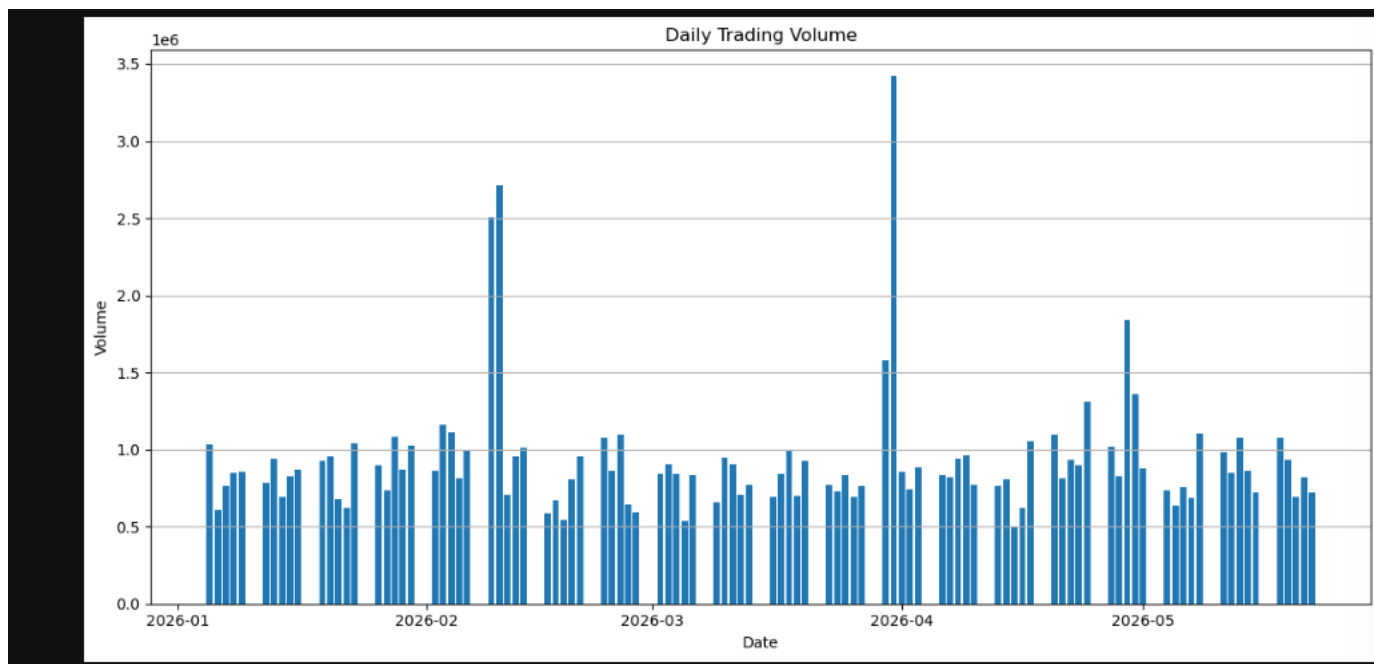
```
plt.xlabel("Date")
```

```
plt.ylabel("Volume")
```

```
plt.grid(True, axis="y")
```

```
plt.tight_layout()
```

```
plt.show()
```



STEP 21 — CREATE GRAPH 4

Closing Price + 20-Day Moving Average

```
plt.figure(figsize=(12, 6))
```

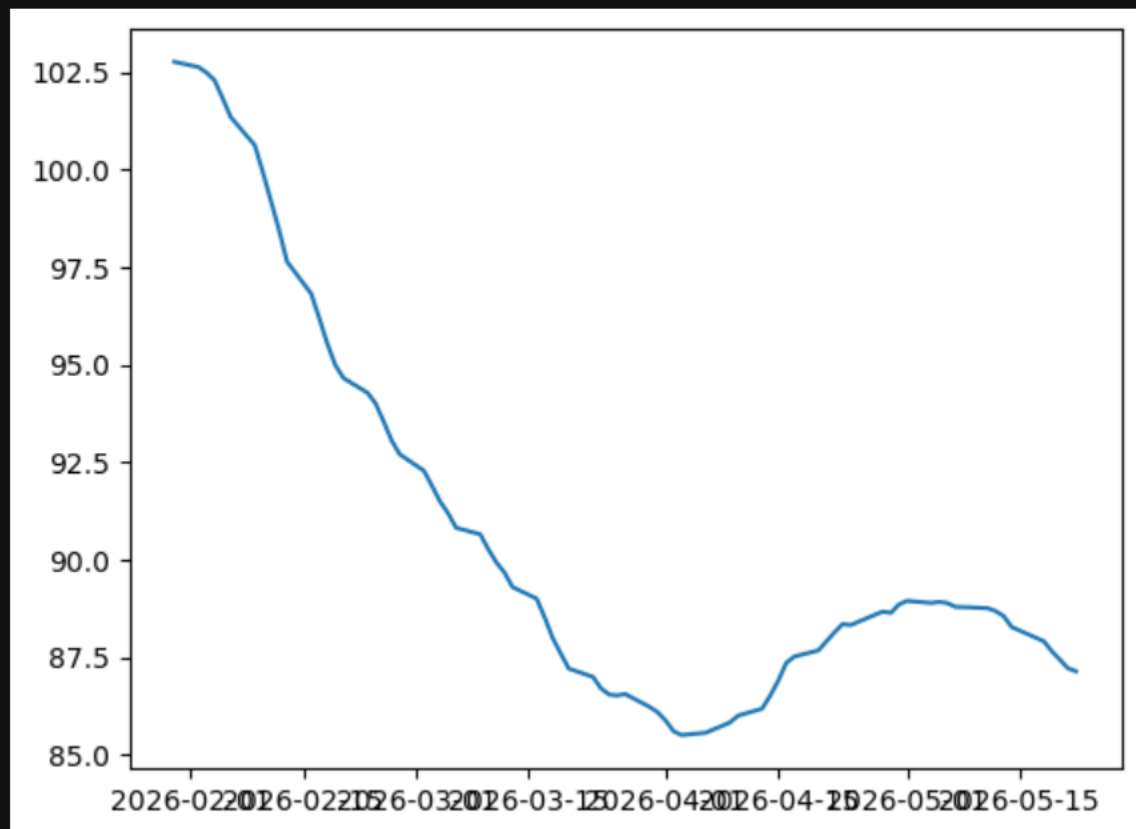
```
plt.plot(  
    df["Date"],  
    df["Close"],  
    label="Close"  
)
```

```
[37]: [<matplotlib.lines.Line2D at 0x19af4ad2fd0>]
```

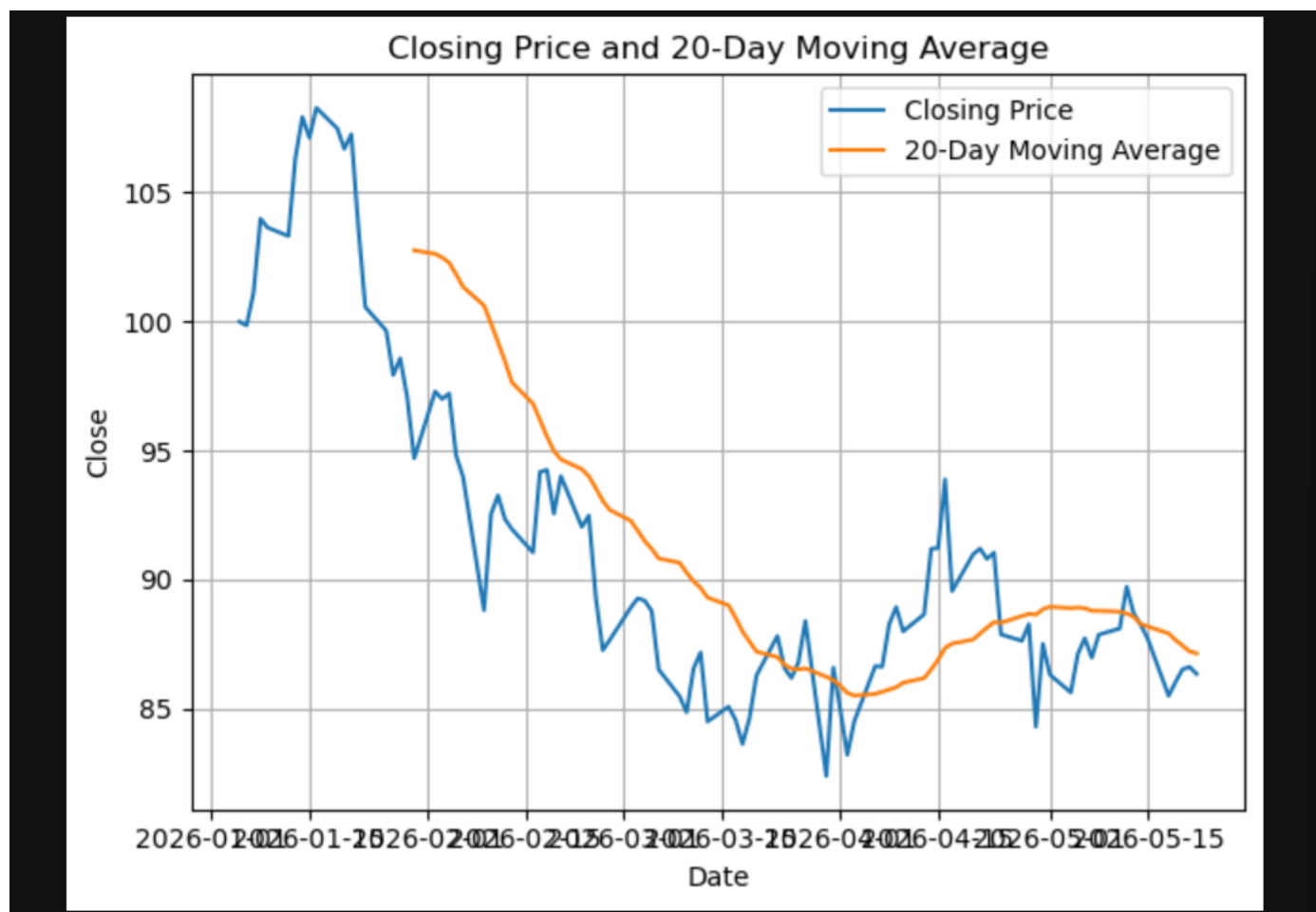


```
plt.plot(  
    df["Date"],  
    df["20_Day_MA"],  
    label="20-Day Moving Average"  
)
```

```
[38]: [<matplotlib.lines.Line2D at 0x19af4b04550>]
```



```
plt.plot(df["Date"], df["Close"], label="Closing Price")
plt.plot(df["Date"], df["20_Day_MA"], label="20-Day Moving Average")
plt.title("Closing Price and 20-Day Moving Average")
plt.xlabel("Date")
plt.ylabel("Close")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

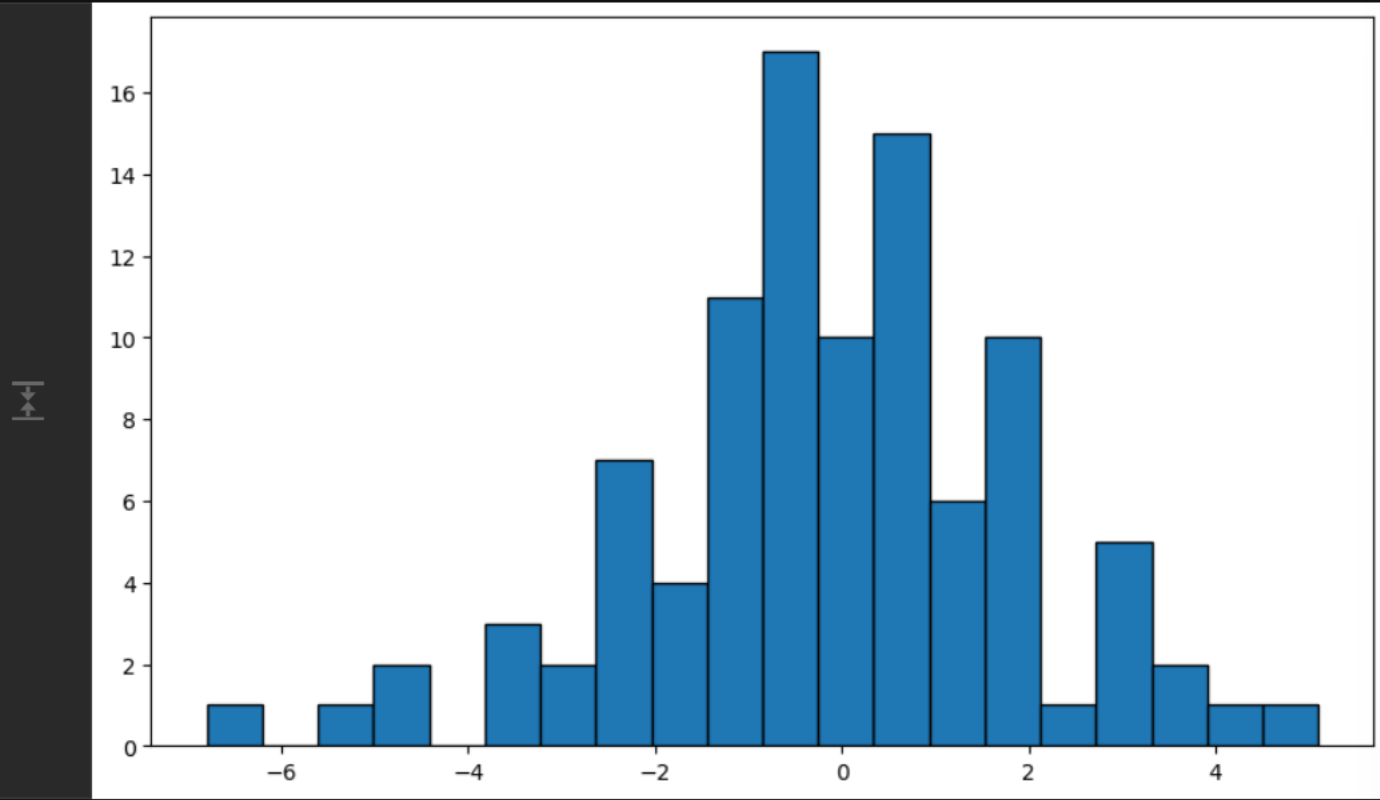


STEP 22 — CREATE GRAPH 5

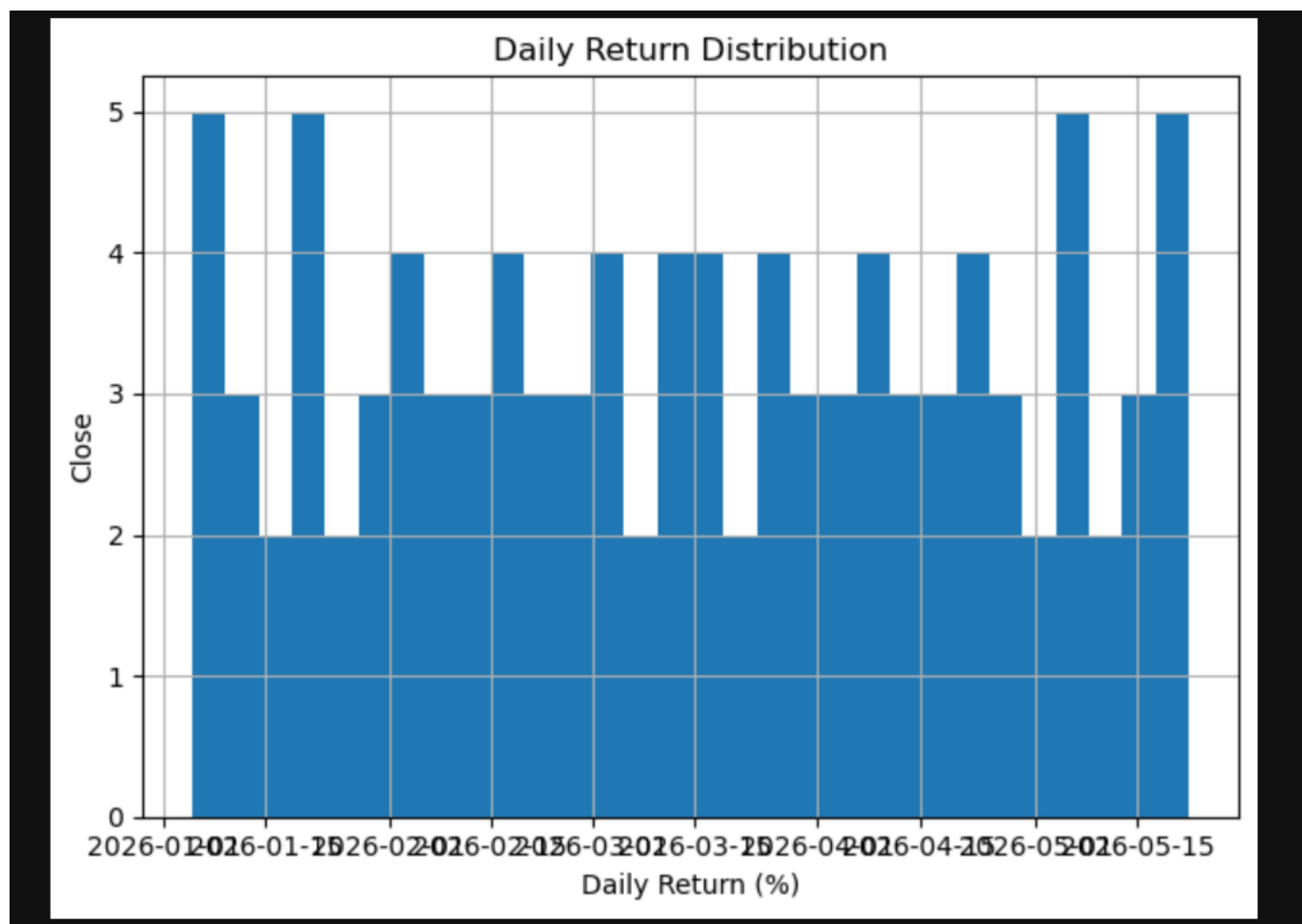
Daily Return Distribution

```
plt.figure(figsize=(10, 6))  
plt.hist(  
    df["Daily_Return_%"].dropna(),  
    bins=20,  
    edgecolor="black"  
)
```

```
[40]: (array([ 1.,  0.,  1.,  2.,  0.,  3.,  2.,  7.,  4., 11., 17., 10., 15.,
        6., 10.,  1.,  5.,  2.,  1.,  1.]),
       array([-6.7994117 , -6.20452487, -5.60963804, -5.01475122, -4.41986439,
        -3.82497756, -3.23009074, -2.63520391, -2.04031708, -1.44543026,
        -0.85054343, -0.2556566 ,  0.33923022,  0.93411705,  1.52900388,
         2.1238907 ,  2.71877753,  3.31366436,  3.90855118,  4.50343801,
         5.09832484])),
       <BarContainer object of 20 artists>)
```



```
plt.hist(df["Date"], bins=30)
plt.title("Daily Return Distribution")
plt.xlabel("Daily Return (%)")
plt.ylabel("Close")
plt.grid(True)
plt.tight_layout()
plt.show()
```



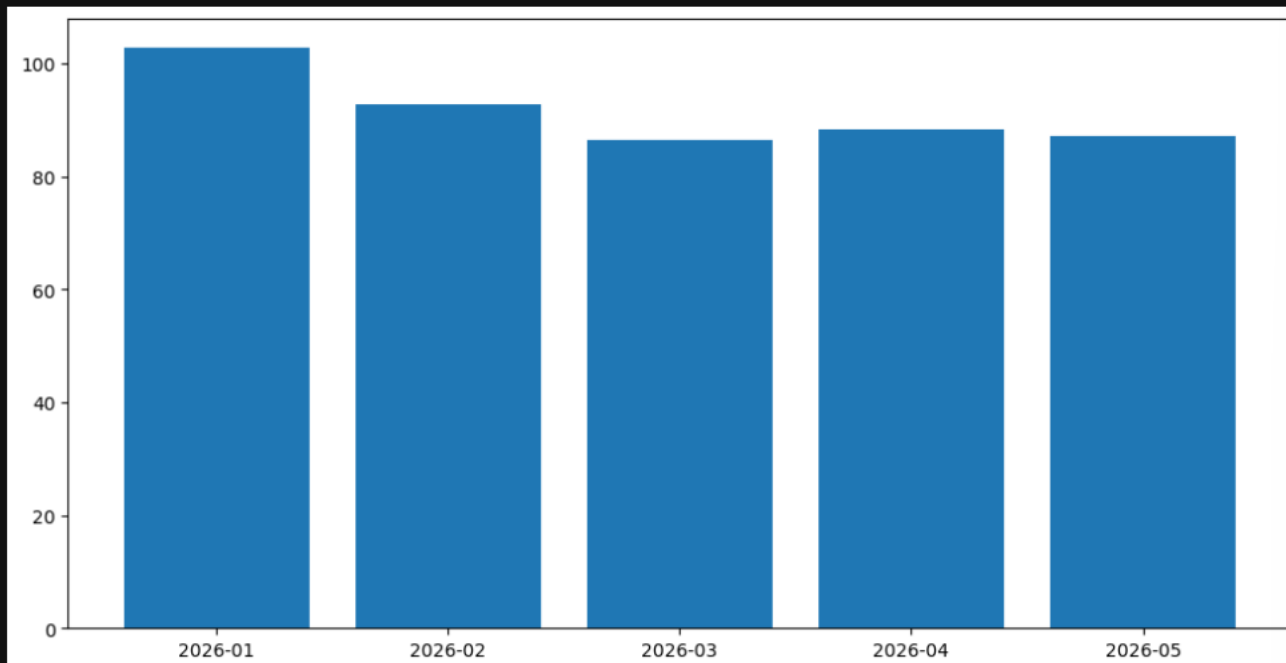
STEP 23 — CREATE GRAPH 6

Average Closing Price by Month

```
plt.figure(figsize=(12, 6))

plt.bar(
    month_summary["Month"],
    month_summary["Average_Close"]
)
```

```
[42]: <BarContainer object of 5 artists>
```



```
plt.figure(figsize=(12, 6))  
plt.bar(month_summary["Month"], month_summary["Average_Close"])  
plt.title("Average Closing Price by Month")  
plt.xlabel("Month")  
plt.ylabel("Average Close")  
plt.xticks(rotation=45)  
plt.grid(True)  
plt.tight_layout()  
plt.show()
```

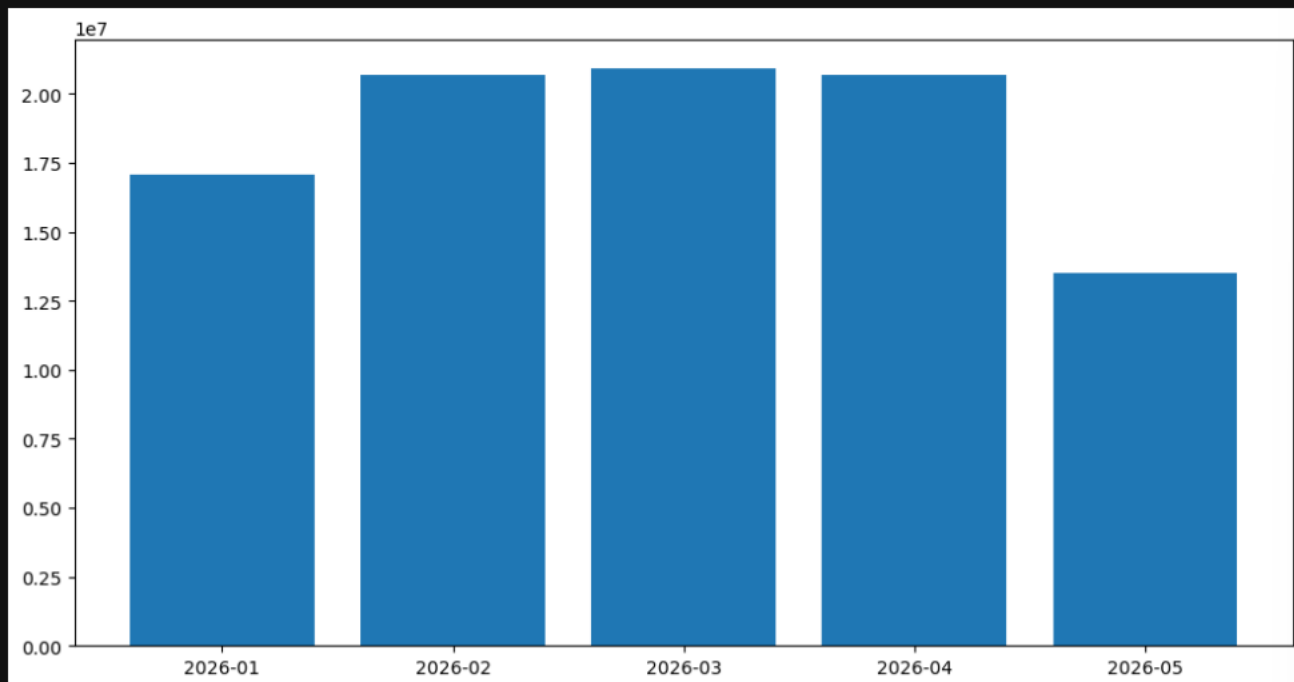


STEP 24 — CREATE GRAPH 7

Total Trading Volume by Month

```
plt.figure(figsize=(12, 6))  
  
plt.bar(  
    month_summary["Month"],  
    month_summary["Total_Volume"]  
)
```

```
[45]: <BarContainer object of 5 artists>
```



```
plt.figure(figsize=(12, 6))
```

```
plt.bar(month_summary["Month"], month_summary["Total_Volume"])
```

```
plt.title("Total Trading Volume by Month")
```

```
plt.xlabel("Month")
```

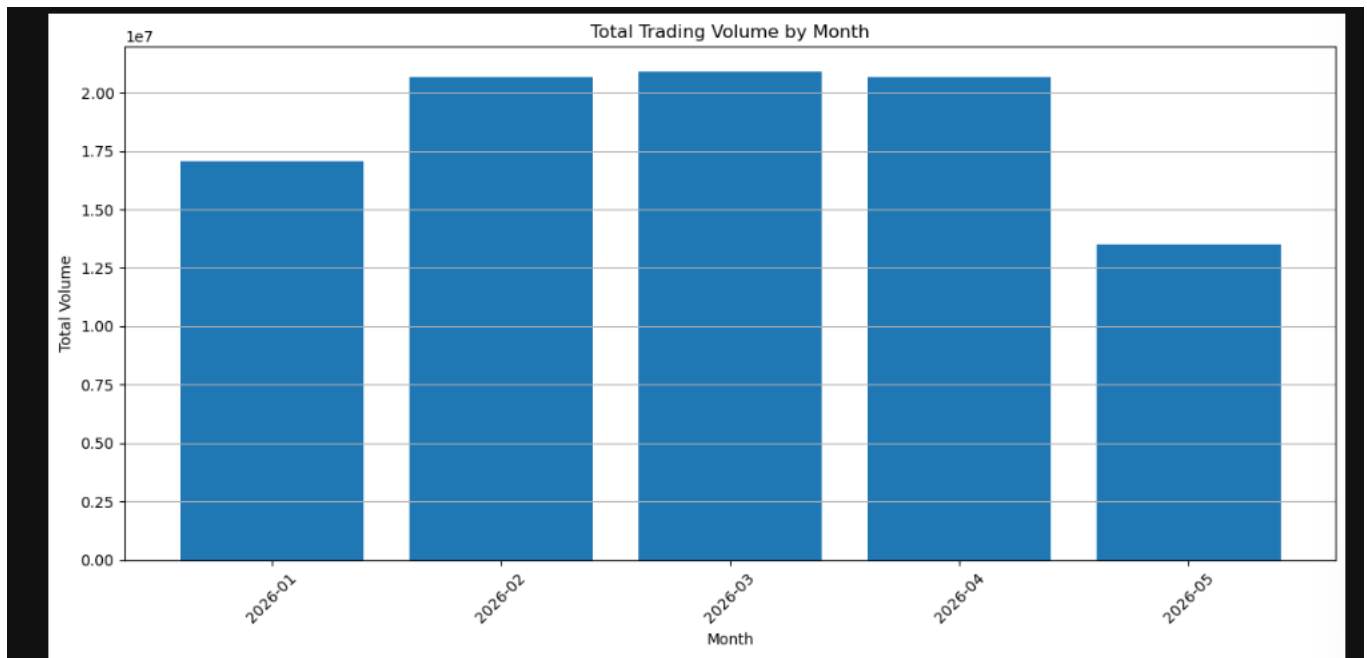
```
plt.ylabel("Total Volume")
```

```
plt.xticks(rotation=45)
```

```
plt.grid(True, axis="y")
```

```
plt.tight_layout()
```

```
plt.show()
```



STEP 25 — CREATE GRAPH 8

Trading Volume vs Closing Price

```
plt.figure(figsize=(10, 6))
```

```
plt.scatter(
```

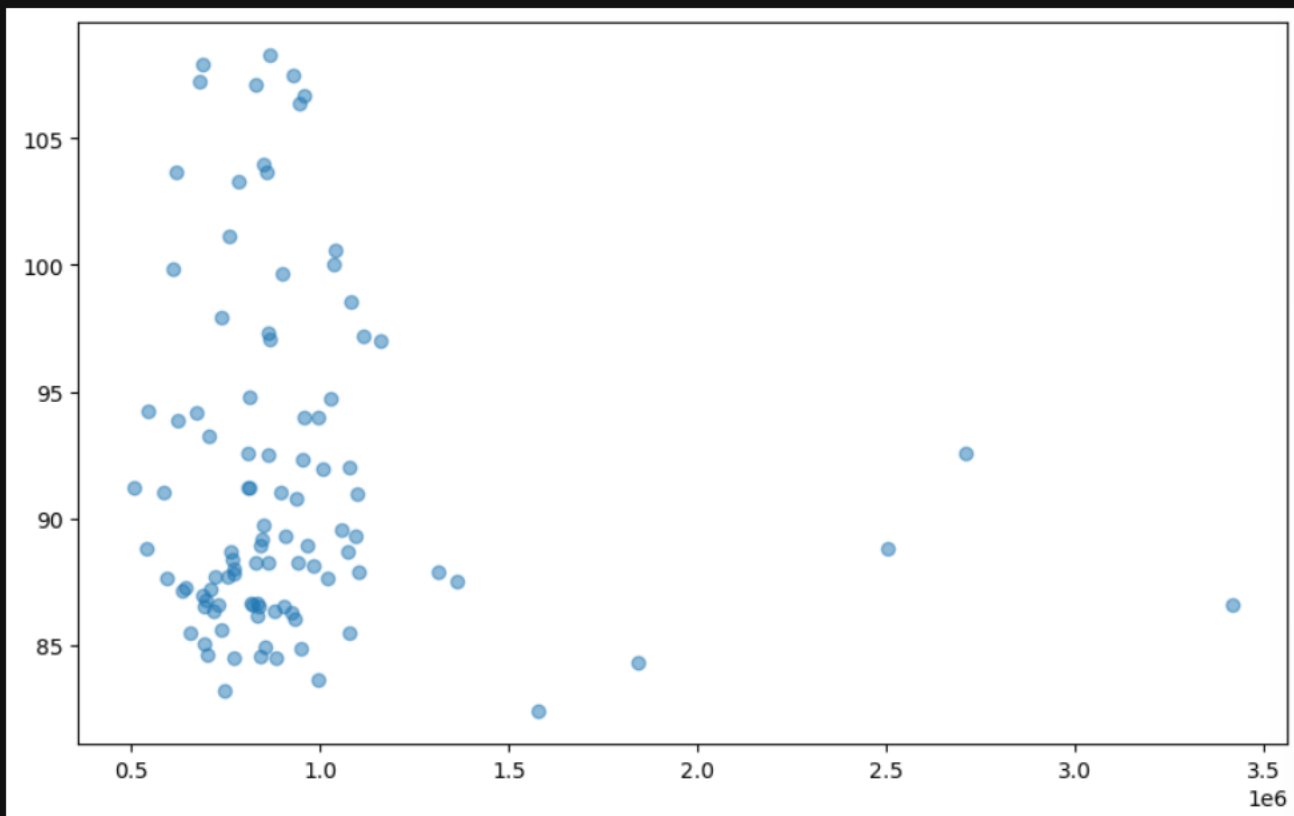
```
    df["Volume"],
```

```
    df["Close"],
```

```
    alpha=0.5
```

```
)
```

```
[48]: <matplotlib.collections.PathCollection at 0x19af4aa8440>
```



```
plt.figure(figsize=(10, 6))  
plt.scatter(df["Volume"], df["Close"], alpha=0.5)  
plt.title("Trading Volume vs Closing Price")  
plt.xlabel("Trading Volume")  
plt.ylabel("Closing Price")  
plt.grid(True)  
plt.tight_layout()  
plt.show()
```



STEP 26 — CORRELATION ANALYSIS

```
numeric_cols = [  
    "Open",  
    "High",  
    "Low",  
    "Close",  
    "Volume",  
    "Daily_Return_%"  
]  
  
correlation_matrix = df[numeric_cols].corr()  
print("--- Correlation Matrix ---")  
print(correlation_matrix)
```

```
--- Correlation Matrix ---
```

	Open	High	Low	Close	Volume	\
Open	1.000000	0.986439	0.980897	0.957016	-0.086782	
High	0.986439	1.000000	0.980403	0.979685	-0.041074	
Low	0.980897	0.980403	1.000000	0.987372	-0.142387	
Close	0.957016	0.979685	0.987372	1.000000	-0.098169	
Volume	-0.086782	-0.041074	-0.142387	-0.098169	1.000000	
Daily_Return_%	-0.164100	-0.042806	-0.002040	0.120418	0.006617	

	Daily_Return_%
Open	-0.164100
High	-0.042806
Low	-0.002040
Close	0.120418
Volume	0.006617
Daily_Return_%	1.000000

```
plt.figure(figsize=(10, 8))
```

```
sns.heatmap(
```

```
    correlation_matrix,
```

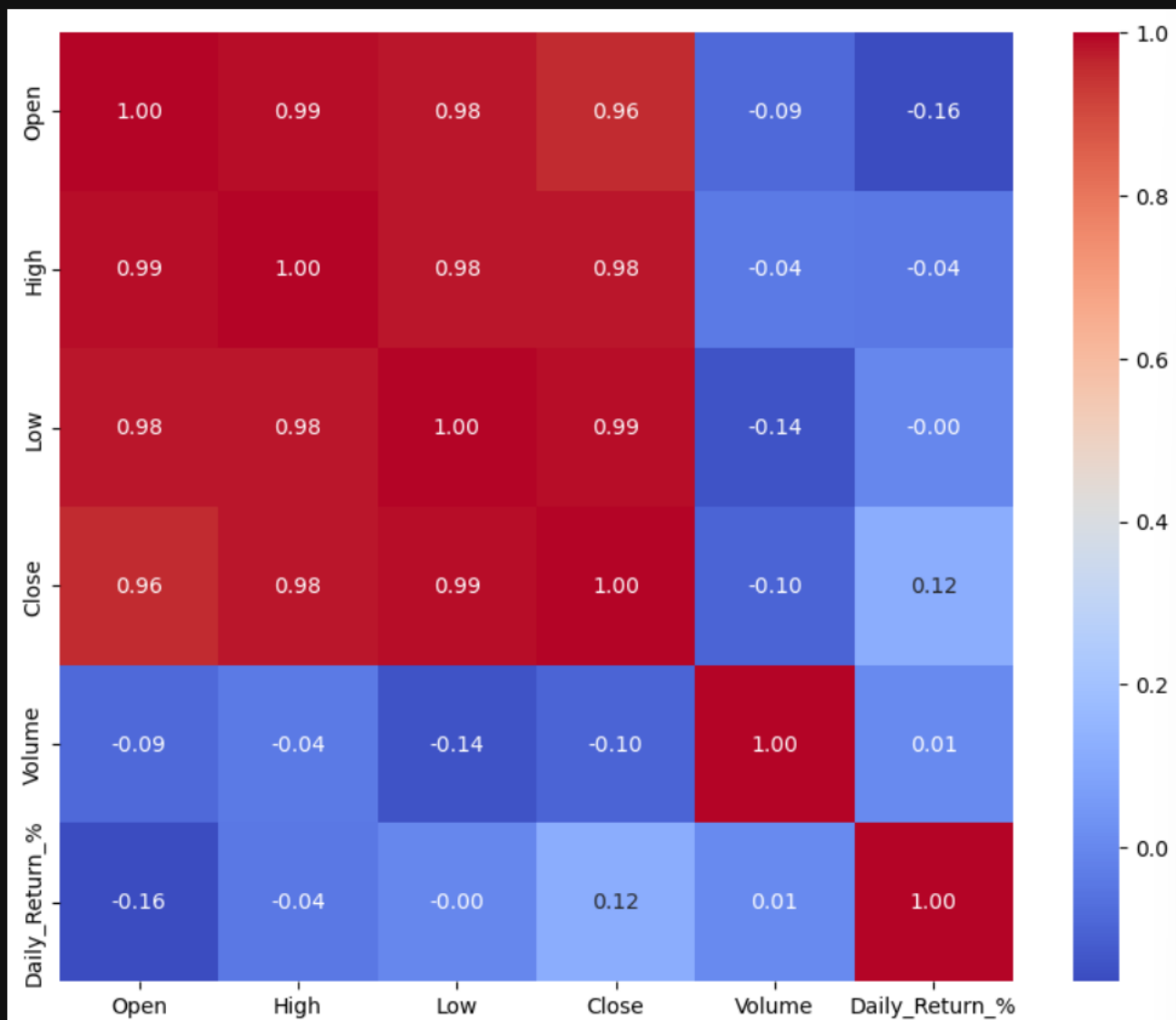
```
    annot=True,
```

```
    cmap="coolwarm",
```

```
    fmt=".2f"
```

```
)
```

[53]: <Axes: >



```
plt.figure(figsize=(10, 8))
```

```
sns.heatmap(correlation_matrix,
```

```
            annot=True,
```

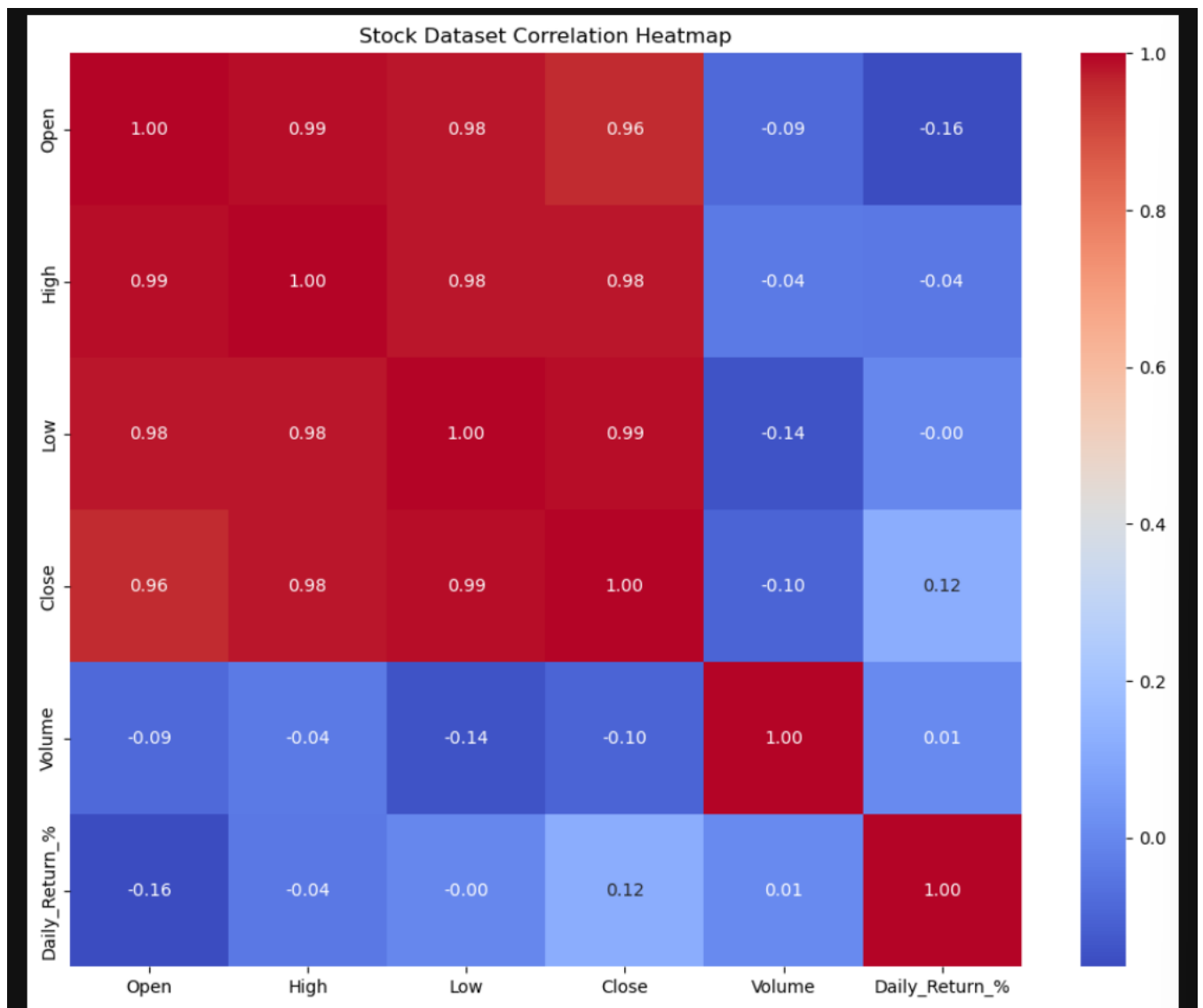
```
            cmap="coolwarm",
```

```
            fmt=".2f")
```

```
plt.title("Stock Dataset Correlation Heatmap")
```

```
plt.tight_layout()
```

```
plt.show()
```



STEP 27 — CREATE DASHBOARD KPIS

```
print("=" * 55)

print(" STOCK FINANCIAL DASHBOARD KPIS")

print("=" * 55)

print("Total Trading Volume:", total_volume)

print("Average Closing Price:", average_close)

print("Highest Closing Price:", highest_close)

print("Lowest Closing Price:", lowest_close)

print("Total Records:", len(df))
```

```
print("Average Daily Volume:", average_volume)
```

```
=====
STOCK FINANCIAL DASHBOARD KPIS
=====
Total Trading Volume: 92840228
Average Closing Price: 91.4946
Highest Closing Price: 108.29
Lowest Closing Price: 82.38
Total Records: 100
Average Daily Volume: 928402.28
```

STEP 28 — FIND THE HIGHEST CLOSING PRICE DATE

```
top_close_date = df.loc[
    df["Close"].idxmax(),
    "Date"
]

print("Highest Closing Price Date:", top_close_date)
print("Highest Closing Price:", highest_close)
```

```
Highest Closing Price Date: 2026-01-16 00:00:00
Highest Closing Price: 108.29
```

STEP 29 — FIND THE HIGHEST VOLUME DATE

```
max_volume_date = df.loc[
    df["Volume"].idxmax(),
    "Date"
]

print("Highest Volume Date:", max_volume_date)
print("Highest Volume:", df["Volume"].max())
```

```
Highest Volume Date: 2026-03-31 00:00:00
Highest Volume: 3418687
```

STEP 30 — FIND BEST AND WORST DAILY RETURN

```
best_day = df.loc[df["Daily_Return_%"].idxmax()]
worst_day = df.loc[df["Daily_Return_%"].idxmin()]

print(
    "Best Daily Return:",
    best_day["Daily_Return_%"],
    "on",
    best_day["Date"]
)

print(
    "Worst Daily Return:",
    worst_day["Daily_Return_%"],
    "on",
    worst_day["Date"]
)
```

```
Best Daily Return: 5.0983248361252675 on 2026-03-31 00:00:00
Worst Daily Return: -6.799411698155911 on 2026-03-30 00:00:00
```

STEP 31 — SAVE CLEANED DATA

```
df.to_csv(
    "stock_cleaned.csv",
    index=False
)

print("Cleaned dataset saved successfully!")
```

```
Cleaned dataset saved successfully!
```

STEP 32 — SAVE THE 3 GROUP-BY SUMMARIES

```
year_summary.to_csv(
```

```
"groupby_year_summary.csv",
index=False
)
month_summary.to_csv(
    "groupby_month_summary.csv",
    index=False
)
quarter_summary.to_csv(
    "groupby_quarter_summary.csv",
    index=False
)
```

Now you have four important output CSV files:

```
stock_cleaned.csv
groupby_year_summary.csv
groupby_month_summary.csv
groupby_quarter_summary.csv
```

STEP 33 — CREATE FINAL SUMMARY

```
final_summary = {
    "Total Records": len(df),
    "Total Trading Volume": total_volume,
    "Average Closing Price": average_close,
    "Highest Closing Price": highest_close,
    "Lowest Closing Price": lowest_close,
    "Average Daily Volume": average_volume,
    "Overall Change (%)": overall_change,
    "Starting Date": df["Date"].min(),
    "Ending Date": df["Date"].max(),
```

```

    "Duplicate Records": df.duplicated().sum(),
    "Missing Values": df.isnull().sum().sum()
}

final_summary

```

```

[66]: {'Total Records': 100,
      'Total Trading Volume': np.int64(92840228),
      'Average Closing Price': np.float64(91.4946),
      'Highest Closing Price': 108.29,
      'Lowest Closing Price': 82.38,
      'Average Daily Volume': np.float64(928402.28),
      'Overall Change (%)': np.float64(-13.659999999999995),
      'Starting Date': Timestamp('2026-01-05 00:00:00'),
      'Ending Date': Timestamp('2026-05-22 00:00:00'),
      'Duplicate Records': np.int64(0),
      'Missing Values': np.int64(20)}

```

STEP 34 — FINAL REPORT PRINT FORMAT

```

print("=" * 60)

print(" CAPSTONE PROJECT FINAL REPORT")

print(" AUTOMATED FINANCIAL DASHBOARD")

print("=" * 60)

print("\n1. DATASET OVERVIEW")

print("-----")

print("Total Records:", len(df))

print("Total Columns:", df.shape[1])

print("Starting Date:", df["Date"].min())

print("Ending Date:", df["Date"].max())

print("\n2. FINANCIAL SUMMARY")

print("-----")

print("Total Trading Volume:", total_volume)

print("Average Closing Price:", average_close)

print("Highest Closing Price:", highest_close)

```

```

print("Lowest Closing Price:", lowest_close)

print("Average Daily Volume:", average_volume)

print("\n3. STOCK ANALYSIS")

print("-----")

print("First Closing Price:", first_close)

print("Last Closing Price:", last_close)

print("Overall Change (%):", overall_change)


print("\n4. RETURN ANALYSIS")

print("-----")

```

```

=====
CAPSTONE PROJECT FINAL REPORT
AUTOMATED FINANCIAL DASHBOARD
=====

1. DATASET OVERVIEW
-----
Total Records: 100
Total Columns: 12
Starting Date: 2026-01-05 00:00:00
Ending Date: 2026-05-22 00:00:00

2. FINANCIAL SUMMARY
-----
Total Trading Volume: 92840228
Average Closing Price: 91.4946
Highest Closing Price: 108.29
Lowest Closing Price: 82.38
Average Daily Volume: 928402.28

3. STOCK ANALYSIS
-----
First Closing Price: 100.0
Last Closing Price: 86.34
Overall Change (%): -13.659999999999995

4. RETURN ANALYSIS
-----

```

```

print(

    "Best Daily Return:",

    best_day["Daily_Return_%"],

```

```

        "%"
    )

    print(
        "Worst Daily Return:",
        worst_day["Daily_Return_%"],
        "%"
    )

    print("\n5. VOLUME ANALYSIS")
    print("-----")

    print("Highest Volume:", df["Volume"].max())
    print("Highest Volume Date:", max_volume_date)

    print("\n6. DATA QUALITY")
    print("-----")

    print("Duplicate Records:", df.duplicated().sum())
    print("Missing Values:", df.isnull().sum().sum())

    print("\n" + "=" * 60)
    print(" END OF FINAL REPORT")
    print("=" * 60)

```

```
Best Daily Return: 5.0983248361252675 %
Worst Daily Return: -6.799411698155911 %
```

```
5. VOLUME ANALYSIS
```

```
-----
```

```
Highest Volume: 3418687
```

```
Highest Volume Date: 2026-03-31 00:00:00
```

```
6. DATA QUALITY
```

```
-----
```

```
Duplicate Records: 0
```

```
Missing Values: 20
```

```
=====
END OF FINAL REPORT
=====
```

STEP 38 — Final results

Actual analysis produced these important values:

Item	Result
Total records	100
Total trading volume	92,840,228
Average closing price	91.49
Highest closing price	108.29
Lowest closing price	82.38
Overall first-to-last change	-13.66%
Missing values	0
Duplicate records	0